Übungsblatt 3: Threads, Präemption und zeitgesteuertes Warten

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Zurückkehren, warten oder weiterrechnen? (6 Punkte)

Eine Sortierstation soll alle 200 ms eine neue Messung bearbeiten.

1. Im kooperativen Programm wird `MessungStep()` aus der Hauptschleife aufgerufen. In einem RTOS-Projekt gibt es stattdessen den Thread `Messung(void *argument)`. Wo darf eine Endlosschleife stehen und wie kommt jeweils die nächste Aufgabe zum Zug? (2 Punkte)
2. Der RTOS-Thread ruft `osDelay(7)` auf. Geben Sie seine Zustandsfolge bis zum nächsten Lauf an. Welche Einheit hat die Sieben und warum garantiert das Ablaufen der Wartezeit noch keinen sofortigen Start? (2 Punkte)
3. Jemand ersetzt den Aufruf durch die übliche `HAL_Delay(7)`. Wird der Thread dadurch Blocked? Können höher priorisierte Threads und zugelassene Interrupts trotzdem ausgeführt werden? (2 Punkte)

Aufgabe 2: Drei endliche Arbeiten auf einem Kern (10 Punkte)

Für eine Paketwaage gelten feste Prioritäten und Präemption. Alle API-, Interrupt- und Wechselkosten sind null. Die Tabelle enthält Freigaben und eigene CPU-Zeiten, keine Boardmessungen.

Thread	Priorität	Freigaben / Verhalten	CPU-Zeit pro Job
Ausschleusen	höchste	wartet bis 3 ms; ein Job, danach wieder warten	2 ms
Wiegen	mittlere	periodisch bei 0, 6, 12 ms; nach jedem Job bis zur nächsten Freigabe warten	2 ms
Etikettbericht	niedrigste	ein Job bei 0; nach Abschluss warten	5 ms

Für Wiegen gilt die relative Deadline $D = 3\,\text{ms}$. Der Ausschleusethread ist bei null bereits Blocked; andere Arbeit fehlt. Leerlauf wird als Idle bezeichnet.

1. Zeichnen Sie die CPU-Belegung im halboffenen Intervall $[0, 15)\,\text{ms}$. Kennzeichnen Sie die Ursachen der Wechsel. (4 Punkte)

2. Nennen Sie die Zustände der drei Threads bei 2,5 und 3,5 ms. (2 Punkte)
3. Berechnen Sie die CPU-Zeitanteile einschließlich Idle in diesem Intervall und die Antwortzeiten aller drei Wiegejobs. (2 Punkte)
4. Ausschleusen erhält zusätzlich einen zweiten Job bei 6 ms. Wann startet und endet der ebenfalls bei 6 ms freigegebene Wiegejob? Hält er seine Deadline? (2 Punkte)

Aufgabe 3: Ein Tickplan mit und ohne Drift (8 Punkte)

1. Ein Kernel hat $f_{\text{tick}} = 250\,\text{Hz}$. Berechnen Sie Tickperiode und aufgerundete Tickzahlen für 6 ms und 18 ms. Warum garantiert `osDelay(1)` keine physische Mindestwartezeit von einer ganzen Tickperiode? (3 Punkte)
2. Für diese Teilaufgabe gilt stattdessen ein 1-kHz-Tick. Drei Arbeitsphasen benötigen nacheinander 3, 1 und 5 ms. Der erste Start ist bei null, alle Arbeitsenden liegen genau auf Tickgrenzen; weitere Störungen fehlen. Bestimmen Sie die nächsten drei Starts bei „Arbeit, dann `osDelay(12)`“ und bei einem absoluten 12-Tick-Sollraster. (3 Punkte)
3. Eine Arbeit endet bei Tick 53; der nächste geplante Start bei Tick 48 ist bereits verpasst. Die Periode beträgt 12 Ticks. Geben Sie eine geeignete Reaktion ohne sofortige Nachholserie an. Was kann ein Zähler verpasster Warteziele aussagen, was ohne zusätzliche Deadlinedefinition nicht? (2 Punkte)

Aufgabe 4: Ressourcen und falsche Abhilfen (8 Punkte)

1. Drei CMSIS-Threads erhalten Stacks mit 768, 1536 und 640 Byte. Pro Thread werden zusätzlich 96 Byte Verwaltungsdaten angenommen. Berechnen Sie die Summe. Welche weiteren Anteile fehlen für eine vollständige Speicherbilanz? Wie viele Byte sind 192 native `StackType_t`-Elemente zu je vier Byte? (3 Punkte)
2. Ein Diagnose-Thread steht über einer Ausgabesteuerung, bleibt ständig Ready und ruft in seiner Rechenschleife `osThreadYield()` auf. Die Ausgabesteuerung bleibt ebenfalls Ready. Wird sie dadurch ausgewählt? Benennen Sie das Problem und eine geeignete Änderung. Es gibt keine gemeinsam gehaltene Ressource. (3 Punkte)
3. Die logische Interruptschwelle für Kernelaufrufe ist 6; vier Prioritätsbits und passende Gruppierung sind vorausgesetzt. Darf ein IRQ mit logischer Priorität 4 `osThreadFlagsSet` aufrufen? Darf ein hierfür zulässiger IRQ mit Priorität 7 `osThreadFlagsWait` aufrufen? (2 Punkte)

Aufgabe 5: Wie viele Hinweise bleiben erhalten?

Ein Thread wartet auf Bit 2, also die Maske `0x04U`. Ein zulässiger anderer Thread setzt dieses Bit bei 2, 3 und 9 ms. Der Empfänger ruft `osThreadFlagsWait(0x04U, osFlagsWaitAny, osWaitForever)` bei 5 und 11 ms und direkt nach dem zweiten erfolgreichen Wait noch einmal auf. Vorher ist das Bit gelöscht, weitere Set-Aufrufe fehlen. Erfolgreiche Waits löschen das angenommene Bit automatisch.

1. Welche beiden Waits kehren sofort zurück und was passiert beim dritten?
2. Wie viele Set-Aufrufe und wie viele erfolgreiche Waits gibt es? Warum sind diese Zahlen verschieden?
3. Wie erkennen Sie einen Fehler im Rückgabewert einer Flagfunktion? Weshalb wäre ein Vergleich mit `osOK` falsch?

Aufgabe 6: HeavyMath, Heartbeat und Tasterreaktion mit FreeRTOS

Programmieren, etwa 35–50 Minuten. Implementieren Sie das Beispiel aus Vorlesung 3 auf dem **NUCLEO-H755ZI-Q / STM32H755ZIT6** mit STM32CubeMX und VS Code mit STM32 VS Code Extension. Es läuft ausschließlich der **M4-Kern**, auch für Systemstart und Taktkonfiguration; M7 bleibt deaktiviert. Die Projektvorbereitung erfolgt vor dem Termin. Benötigt werden nur die eingebauten LEDs und B1.

Threadfunktion	CMSIS-Priorität	Gefordertes Verhalten
`Lab03_Event`	`osPriorityHigh`	auf Tasterhinweis warten, LD1/PB0 umschalten
`Lab03_Heartbeat`	`osPriorityNormal`	LD2/PE1 sofort, dann alle 500 Kernelticks umschalten
`Lab03_HeavyMath`	`osPriorityBelowNormal`	ununterbrochen rechnen, ständig Ready bleiben

Die vorbereitete Basis nutzt **FreeRTOS, CMSIS-RTOS2, Präemption und 32-Bit-Ticks mit 1000 Hz**. `lab03_rtos.c/.h` erzeugt die drei Threads und stellt GPIO-, IRQ- und Wartehilfen bereit. Implementieren Sie die drei Funktionsrümpfe in `code/lab03/threads_student.c` selbst; entfernen Sie anschließend `WaitForImplementation`.

1. **Heartbeat:** Initialisieren Sie ein lokales `next` mit `osKernelGetTickCount()`. Rufen Sie in einer Endlosschleife `Lab03_ToggleHeartbeat()` und danach `Lab03_WaitHeartbeat(&next)` auf. Die Wartehilfe blockiert bis zum nächsten absoluten 500-Tick-Ziel und richtet sich bei einer Zielverfehlung neu aus. Welche vollständige Blinkperiode ergibt sich im ungestörten Modell? Warum darf hier kein `HAL_Delay` stehen?
2. **HeavyMath:** Aktualisieren Sie in einer Endlosschleife den vorbereiteten `volatile uint32_t lab03_accumulator` mit `lab03_accumulator * 1664525U + 1013904223U`. Verwenden Sie weder Delay noch Yield oder Interruptsperre. Die einfache Rechnung dient als dauerhafte CPU-Last; ihr Ergebnis wird fachlich nicht benötigt. Warum können Heartbeat und Tasterreaktion trotzdem ausgeführt werden?
3. **Tasterreaktion:** Rufen Sie einmal `Lab03_EnableButtonIRQ()` auf. Warten Sie anschließend in einer Endlosschleife mit `osThreadFlagsWait(LAB03_BUTTON_FLAG, osFlagsWaitAny, osWaitForever)`. Prüfen Sie das Fehlerbit `osFlagsError`; im Fehlerfall `Lab03_Fail()`. Erhöhen Sie nach jedem erfolgreichen Wait `lab03_event_responses` und schalten Sie mit `Lab03_ToggleEvent()` LD1 um. Zwischen Hinweisen soll der Thread Blocked sein.
4. **IRQ-Weg erklären:** B1/PC13 löst über die vorbereitete HAL-Callbackkette `Lab03_ButtonIRQ()` aus; diese setzt das Thread-Flag. Beschreiben Sie den Weg vom Tastendruck bis zur LED-Aktion. Warum setzt die ISR nur einen Hinweis und warum wird der IRQ erst nach dem Kernelstart im Ereignis-Thread freigegeben?

5. **Prüfen:** Bauen und laden Sie nur das CM4-Ziel. Prüfen Sie die folgenden Fälle und protokollieren Sie Erwartung und Beobachtung. Lesen Sie Diagnosewerte gemeinsam nur bei angehaltenem M4; ein Debughalt ist keine Zeitmessung.

Versuch	Zu prüfende Beobachtung
Grundzustand ohne B1	LD2 blinkt; `lab03_accumulator` verändert sich trotz Heartbeat weiter
B1 während HeavyMath	LD1 reagiert; `lab03_event_responses` steigt; LD2 blinkt weiter; `lab03_irq_errors` bleibt null
HeavyMath höher priorisieren	Vorhersage aus Teilaufgabe 6 mit dem Board vergleichen

6. **Starvation selbst erzeugen:** Ändern Sie ausschließlich die HeavyMath-Priorität in `Lab03_Start()` von `osPriorityBelowNormal` auf `osPriorityAboveNormal`. Ereignis-Thread bleibt `osPriorityHigh`. Sagen Sie vor dem Neustart voraus, was mit beiden LEDs passiert. Begründen Sie, weshalb auch ein `osThreadYield()` in HeavyMath den Heartbeat nicht zuverlässig zum Zug bringt. Stellen Sie danach die ursprüngliche Priorität wieder her.

Ergebnis: drei eigene Threadfunktionen, Prüfprotokoll und Erklärung der Präemption. Tasterprellen kann mehrere Flanken erzeugen; ein Thread-Flag speichert nur einen ausstehenden Hinweis. Ein Tastendruck ist daher weder eine garantierte einzelne Umschaltung noch ein verlustfrei gezähltes Ereignis. Eine Entprellung ist hier kein Pflichtteil. Ohne Boardlauf dessen Status angeben.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; Arm CMSIS-RTOS2; ST UM2408, RM0399 und STM32CubeH7, Template *BootCM4_CM7Gated* (hier ohne Freigabe von CM7). Papierzeiten und Speichergrößen sind Lehrannahmen.